Project 2: Financial Analytics

Project Title: Investment Risk & Volatility Monitor
Product Brand Name: "AlphaPulse
WEEK-1 Data Acquisition & Cleaning
Planning: Select a diverse portfolio of 10 stocks spanning multiple sectors, plus a major index (e.g., S&P 500). Build a robust Python scraper to fetch historical data, including resilient error handling for API rate limits
Data Quality Check: Ensure proper handling of complex financial adjustments like stock splits and dividend payouts.

```
import yfinance as yf
```

```
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
import numpy as np
%matplotlib inline
```

```
symbol =["AAPL","MSFT","JPM","JNJ","XOM","PG","TSLA","DIS","BAC","NVDA","SPY"]
start_date ="2023-01-01"
end_date = "2024-12-24"
data = yf.download(symbol,start=start_date,end = end_date)
```

```
[**********100%**********]  11 of 11 completed
```

```
data.head()
```

Price									
Ticker	AAPL	BAC	DIS	JNJ	JPM	MSFT	NVDA	PG	SPY
Date									
2023-01-03	123.211205	30.974272	86.923019	162.655853	124.928696	233.985641	14.300683	140.514481	366.069092
2023-01-04	124.482040	31.556595	89.863770	164.426758	126.093697	223.750381	14.734250	141.126312	368.895233
2023-01-05	123.161942	31.491896	89.805138	163.212723	126.065735	217.118881	14.250735	139.374146	364.684845
2023-01-06	127.693581	31.806168	91.759132	164.536316	128.478058	219.677719	14.844140	142.693054	373.047882
2023-01-09	128.215729	31.325516	92.589569	160.273422	127.947166	221.816589	15.612371	140.950150	372.836334

5 rows × 55 columns

```
returns = data['Close'].pct_change()
returns.head()
```

Ticker	AAPL	BAC	DIS	JNJ	JPM	MSFT	NVDA	PG	SPY	TSL
Date										
2023-01-03	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	Nal
2023-01-04	0.010314	0.018800	0.033832	0.010887	0.009325	-0.043743	0.030318	0.004354	0.007720	0.05124
2023-01-05	-0.010605	-0.002050	-0.000652	-0.007383	-0.000222	-0.029638	-0.032816	-0.012416	-0.011414	-0.02903
2023-01-06	0.036794	0.009979	0.021758	0.008110	0.019135	0.011785	0.041640	0.023813	0.022932	0.02465
2023-01-09	0.004089	-0.015112	0.009050	-0.025909	-0.004132	0.009736	0.051753	-0.012214	-0.000567	0.05934

```
volatility = returns.std()

volatility
```

```
Ticker
AAPL    0.013474
BAC     0.015731
DIS     0.016514
JNJ     0.009999
JPM     0.014004
MSFT    0.014307
NVDA    0.031916
PG      0.009434
SPY     0.008064
TSLA    0.036619
XOM     0.014037
dtype: float64
```

```
#portfolio volatility

portfolio_returns = returns.mean(axis=1)
portfolio_volatility = portfolio_returns.std()

print("Portfolio Volatility:", round(portfolio_volatility, 4))
```

```
Portfolio Volatility: 0.0092
```

```
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

start_date = "2023-01-01"
end_date = "2024-01-01"

stocks = ["AAPL","MSFT","JPM","JNJ","XOM","PG","TSLA","DIS","BAC","NVDA","SPY"]

# Stock data
data = yf.download(stocks, start=start_date, end=end_date)

# Market data (S&P 500)
market_data = yf.download("^GSPC", start=start_date, end=end_date)

market_data.head()
```

```
[**********100%**********]  11 of 11 completed
[**********100%**********]  1 of 1 completed
```

Price	Close	High	Low	Open	Volume
Ticker	^GSPC	^GSPC	^GSPC	^GSPC	^GSPC
Date					
2023-01-03	3824.139893	3878.459961	3794.330078	3853.290039	3959140000
2023-01-04	3852.969971	3873.159912	3815.770020	3840.360107	4414080000
2023-01-05	3808.100098	3839.739990	3802.419922	3839.739990	3893450000

```
# Daily stock returns
returns = data['Close'].pct_change().dropna()

# Market returns
market_returns = market_data['Close'].pct_change().dropna()

print("Stock data shape:", data.shape)
print("Returns shape:", returns.shape)
print("Market returns shape:", market_returns.shape)
```

```
Stock data shape: (250, 55)
Returns shape: (249, 11)
Market returns shape: (249, 1)
```

```
market_data = yf.download(
    "^GSPC",
    start="2023-01-01",
    end="2024-12-24"
)

market_returns = market_data['Close'].pct_change()

market_returns.head()
```

```
[**********100%**********]  1 of 1 completed
```

Ticker	^GSPC
Date	
2023-01-03	NaN
2023-01-04	0.007539
2023-01-05	-0.011646
2023-01-06	0.022841
2023-01-09	-0.000768

```python
#portfolio returns and market returns

comparison_df = pd.concat(
    [portfolio_returns, market_returns],
    axis=1
)

comparison_df.columns = ["Portfolio_Returns", "Market_Returns"]
comparison_df.dropna(inplace=True)

comparison_df.head()
```

Date	Portfolio_Returns	Market_Returns
2023-01-04	0.012361	0.007539
2023-01-05	-0.010351	-0.011646
2023-01-06	0.021153	0.022841
2023-01-09	0.005219	-0.000768
2023-01-10	0.005911	0.006978

```python
# portfolio annual returns and volatility
annual_return = returns.mean() * 252
annual_volatility = returns.std() * np.sqrt(252)

portfolio_annual_return = portfolio_returns.mean() * 252
portfolio_annual_volatility = portfolio_returns.std() * np.sqrt(252)

print("Portfolio Annual Return:", round(portfolio_annual_return, 4))
print("Portfolio Annual Volatility:", round(portfolio_annual_volatility, 4))
```

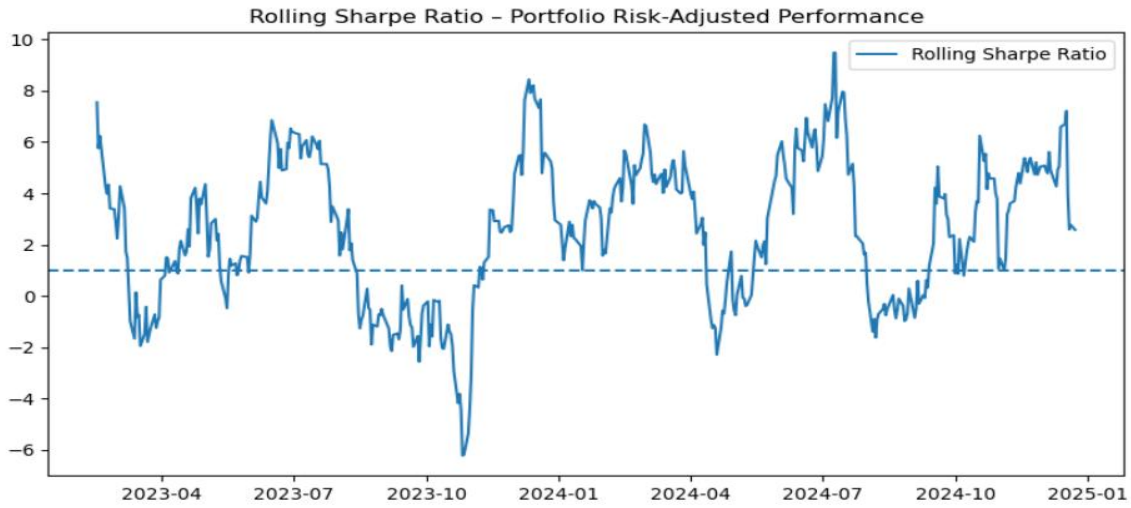
```
Portfolio Annual Return: 0.3554
Portfolio Annual Volatility: 0.1456
```

```python
#portfolio sharpe ratio
risk_free_rate = 0  # project standard

portfolio_sharpe = (
    (portfolio_annual_return - risk_free_rate)
    / portfolio_annual_volatility
)

print("Portfolio Sharpe Ratio:", round(portfolio_sharpe, 3))
```

```
Portfolio Sharpe Ratio: 2.441
```

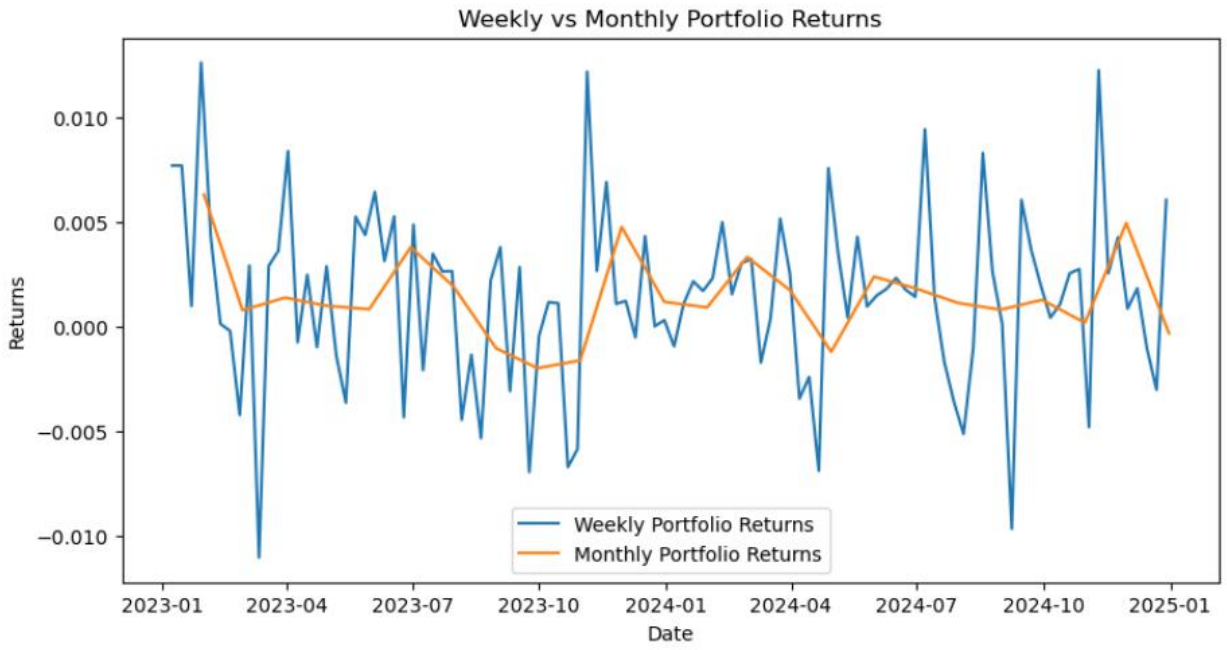
```python
# 30-day rolling mean return
rolling_30d_return = portfolio_returns.rolling(window=30).mean()

# 30-day rolling volatility (std dev)
rolling_30d_volatility = portfolio_returns.rolling(window=30).std()

olling_sharpe = (rolling_30d_return / rolling_30d_volatility) * np.sqrt(252)
```

```python
#rolling sharpe ratio
rolling_sharpe = (
    rolling_30d_return / rolling_30d_volatility
) * np.sqrt(252)

plt.figure(figsize=(10,5))
plt.plot(rolling_sharpe, label="Rolling Sharpe Ratio")
plt.axhline(1, linestyle='--')
plt.legend()
plt.title("Rolling Sharpe Ratio - Portfolio Risk-Adjusted Performance")
plt.show()
```

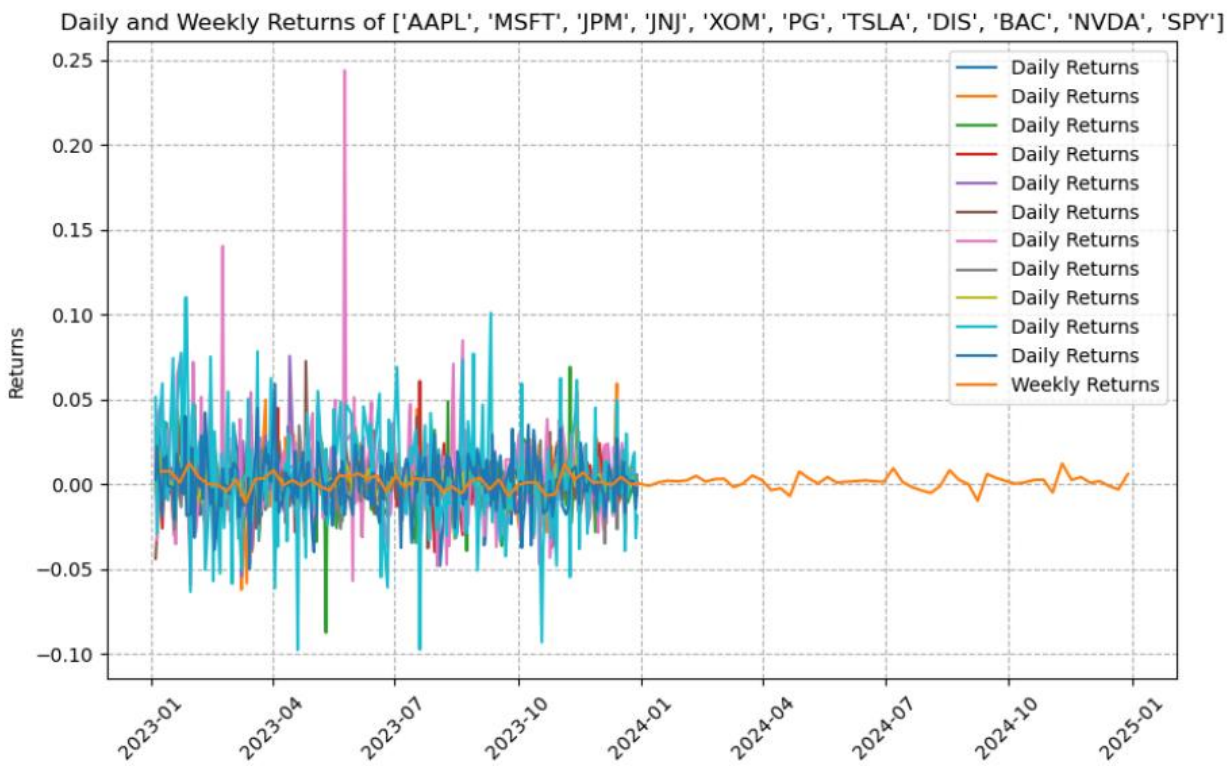


```python
# portfolio beta and alpha

aligned_data = pd.concat(
    [portfolio_returns, market_returns],
    axis=1
).dropna()

aligned_data.columns = ["Portfolio", "Market"]

cov_pm = np.cov(
    aligned_data["Portfolio"],
    aligned_data["Market"]
)[0, 1]

portfolio_beta = cov_pm / aligned_data["Market"].var()

print("Portfolio Beta:", round(portfolio_beta, 3))


market_annual_return = market_returns.mean() * 252

portfolio_alpha = (
    portfolio_annual_return
    - portfolio_beta * market_annual_return
)

print("Portfolio Alpha:", round(portfolio_alpha, 4))
```

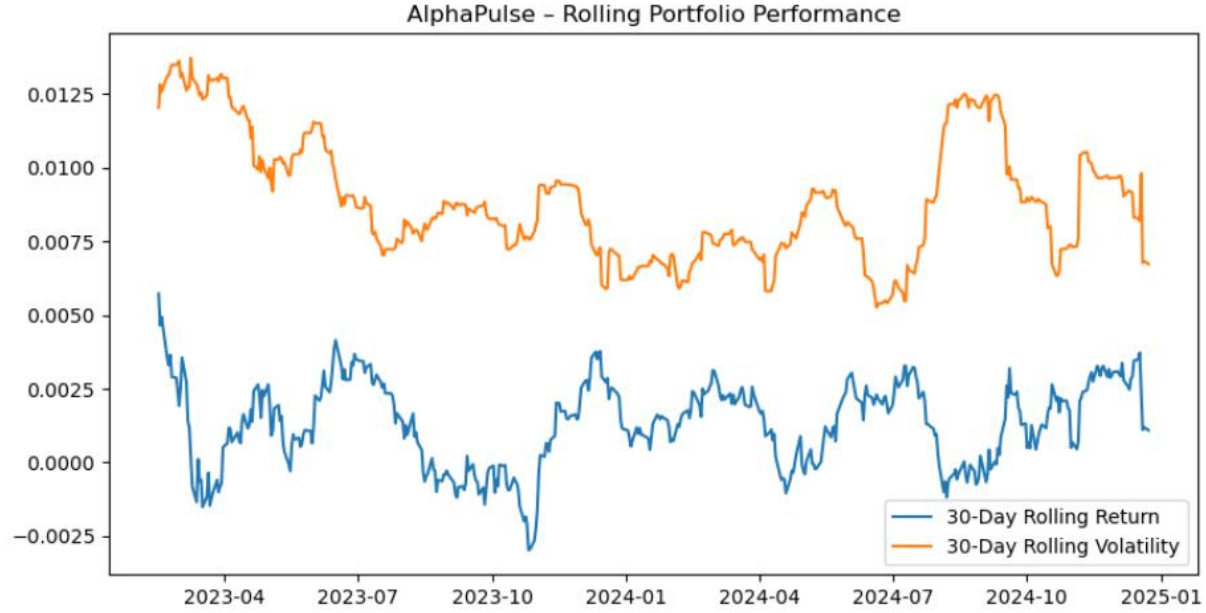
```python
#weekly vs monthly portfolio returns
weekly_returns = portfolio_returns.resample('W').mean()
monthly_returns = portfolio_returns.resample('ME').mean()

plt.figure(figsize=(10, 5))

plt.plot(weekly_returns, label="Weekly Portfolio Returns")
plt.plot(monthly_returns, label="Monthly Portfolio Returns")

plt.legend()
plt.title("Weekly vs Monthly Portfolio Returns")
plt.xlabel("Date")
plt.ylabel("Returns")

plt.show()
```



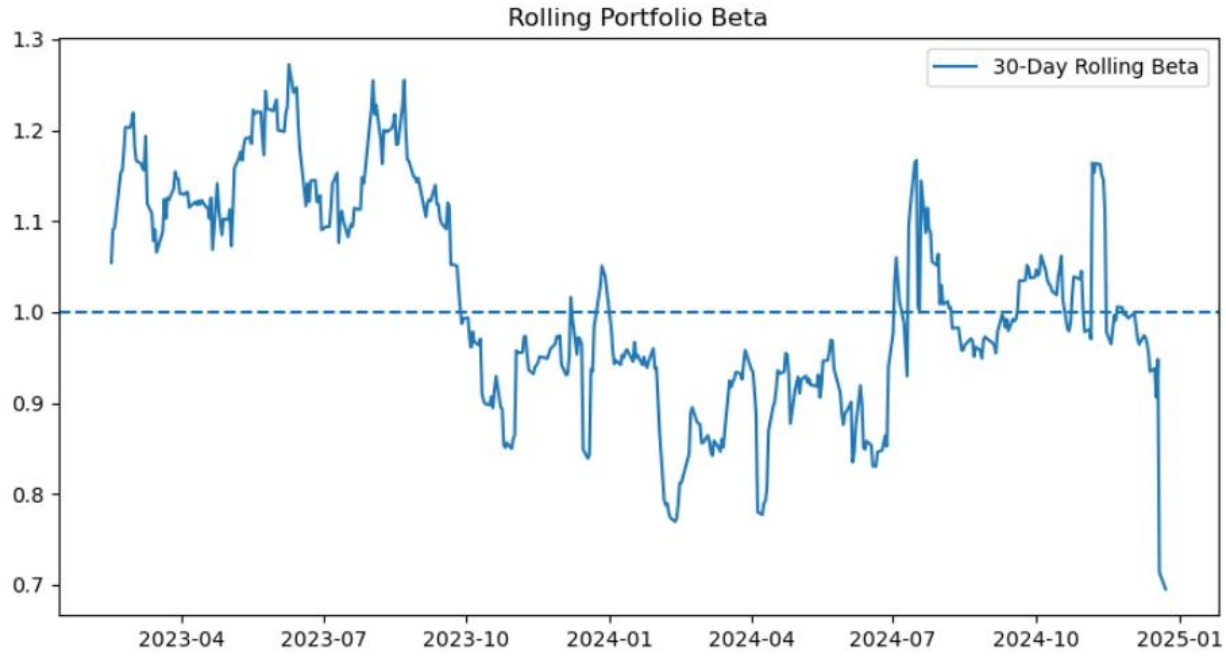

```python
# visualization
plt.figure(figsize=(10,6))

# plotting returns
plt.plot(returns.index, returns, label="Daily Returns")
plt.plot(weekly_returns.index, weekly_returns, label="Weekly Returns")

plt.title(f"Daily and Weekly Returns of {symbol}")
plt.ylabel("Returns")
plt.grid(linestyle="--")
plt.xticks(rotation=45)
plt.legend()
plt.show()
```




```python
#alpha pulse rolling portfolio performance

plt.figure(figsize=(10,5))
plt.plot(rolling_30d_return, label="30-Day Rolling Return")
plt.plot(rolling_30d_volatility, label="30-Day Rolling Volatility")
plt.legend()
plt.title("AlphaPulse – Rolling Portfolio Performance")
plt.show()
```



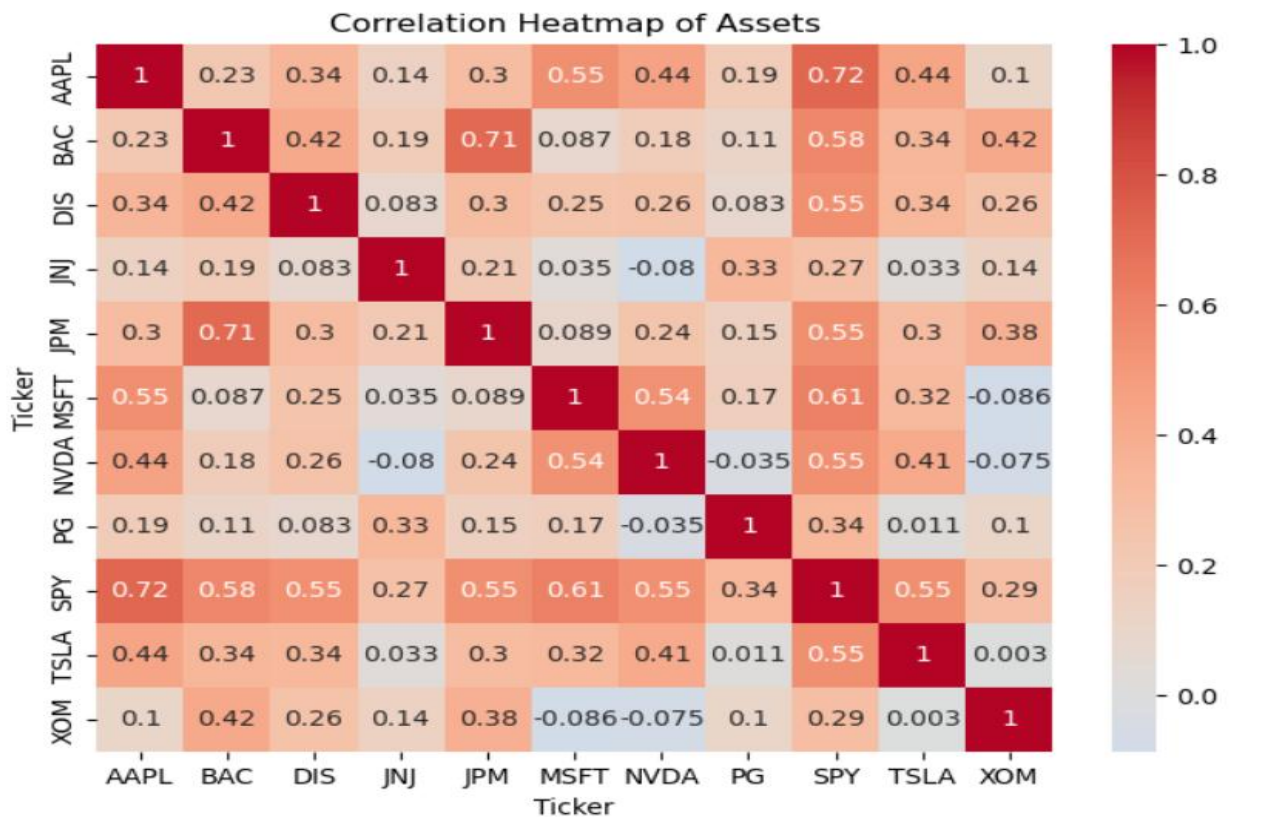
```python
# rolling portfolio beta

aligned = pd.concat(
    [portfolio_returns, market_returns],
    axis=1
).dropna()

aligned.columns = ["Portfolio", "Market"]

rolling_beta = (
    aligned["Portfolio"]
    .rolling(30)
    .cov(aligned["Market"])
    / aligned["Market"].rolling(30).var()
)

plt.figure(figsize=(10,5))
plt.plot(rolling_beta, label="30-Day Rolling Beta")
plt.axhline(1, linestyle='--')
plt.legend()
plt.title("Rolling Portfolio Beta")
plt.show()
```



```
#maximum drawdown
cumulative_returns = (1 + portfolio_returns).cumprod()
peak = cumulative_returns.cummax()
drawdown = (cumulative_returns - peak) / peak
max_drawdown = drawdown.min()

print("Maximum Drawdown:", round(max_drawdown, 4))

#sharpe ratio
portfolio_volatility = portfolio_returns.std() * np.sqrt(252)

sharpe_ratio = (portfolio_annual_return - risk_free_rate) / portfolio_volatility
print("Sharpe Ratio:", round(sharpe_ratio, 3))

#print results
print("volatility:",volatility)
print("beta:",beta)
print("sharpe ratio:",sharpe_ratio)
#print additional results
print("VaR at 95% confidence level:", VaR)
print("alpha:",alpha)
print("treynor ratio:",treynor_ratio)
print("maximum drawdown:",max_drawdown)
```

```
Value at Risk (95%): -0.0142
Beta: 1.02
Alpha: 0.1158
Treynor Ratio: 0.329
Maximum Drawdown: -0.1104
Sharpe Ratio: 2.304
volatility: Ticker
AAPL    0.013474
BAC     0.015731
DIS     0.016514
JNJ     0.009999
JPM     0.014004
MSFT    0.014307
NVDA    0.031916
PG      0.009434
SPY     0.008064
TSLA    0.036619
XOM     0.014037
dtype: float64
beta: 1.0195396428093502
sharpe ratio: 2.3039348657048313
```

```
import seaborn as sns

corr_matrix = returns.corr()

plt.figure(figsize=(8,6))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", center=0)
plt.title("Correlation Heatmap of Assets")
plt.show()
```


Correlation Heatmap of Assets
Ticker
AAPL 1 0.23 0.34 0.14 0.3 0.55 0.44 0.19 0.72 0.44 0.1
BAC 0.23 1 0.42 0.19 0.71 0.087 0.18 0.11 0.58 0.34 0.42
DIS 0.34 0.42 1 0.083 0.3 0.25 0.26 0.083 0.55 0.34 0.26
JNJ 0.14 0.19 0.083 1 0.21 0.035 -0.08 0.33 0.27 0.033 0.14
JPM 0.3 0.71 0.3 0.21 1 0.089 0.24 0.15 0.55 0.3 0.38
MSFT 0.55 0.087 0.25 0.035 0.089 1 0.54 0.17 0.61 0.32 -0.086
NVDA 0.44 0.18 0.26 -0.08 0.24 0.54 1 -0.035 0.55 0.41 -0.075
PG 0.19 0.11 0.083 0.33 0.15 0.17 -0.035 1 0.34 0.011 0.1
SPY 0.72 0.58 0.55 0.27 0.55 0.61 0.55 0.34 1 0.55 0.29
TSLA 0.44 0.34 0.34 0.033 0.3 0.32 0.41 0.011 0.55 1 0.003
XOM 0.1 0.42 0.26 0.14 0.38 -0.086 -0.075 0.1 0.29 0.003 1
AAPL BAC DIS JNJ JPM MSFT NVDA PG SPY TSLA XOM
Ticker
1.0
0.8
0.6
0.4
0.2
0.0

Week 2: Quantitative Analysis

Planning: Calculate Daily Log Returns using efficient NumPy array operations. Develop and implement the Monte Carlo simulation (10,000 runs) to predict the portfolio's value 1 year into the future

Statistical Validation: Validate the Monte Carlo model's output distribution (e.g., skewness, kurtosis) against historical market behavior'

```
import yfinance as yf
```

```
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
import numpy as np
%matplotlib inline
```

```
symbol =["AAPL","MSFT","JPM","JNJ","XOM","PG","TSLA","DIS","BAC","NVDA","SPY"]
start_date ="2023-01-01"
end_date = "2024-12-24"
data = yf.download(symbol,start=start_date,end = end_date)
```

```
[**********100%**********]  11 of 11 completed
```

```
data.head()
```

Price									
Ticker	AAPL	BAC	DIS	JNJ	JPM	MSFT	NVDA	PG	SPY
Date									
2023-01-03	123.211205	30.974268	86.923019	162.655869	124.928688	233.985657	14.300682	140.514465	366.069092
2023-01-04	124.482033	31.556599	89.863777	164.426773	126.093674	223.750366	14.734249	141.126312	368.895233
2023-01-05	123.161942	31.491890	89.805138	163.212692	126.065758	217.118912	14.250734	139.374161	364.684845
2023-01-06	127.693604	31.806162	91.759125	164.536316	128.478058	219.677734	14.844141	142.693085	373.047821
2023-01-09	128.215744	31.325520	92.589569	160.273407	127.947144	221.816589	15.612370	140.950180	372.836365

```
returns = data['Close'].pct_change()
returns.head()
```

Ticker	AAPL	BAC	DIS	JNJ	JPM	MSFT	NVDA	PG	SPY	TS
Date										
2023-01-03	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	N
2023-01-04	0.010314	0.018800	0.033832	0.010887	0.009325	-0.043743	0.030318	0.004354	0.007720	0.0512
2023-01-05	-0.010605	-0.002051	-0.000653	-0.007384	-0.000221	-0.029638	-0.032816	-0.012415	-0.011414	-0.0290
2023-01-06	0.036794	0.009979	0.021758	0.008110	0.019135	0.011785	0.041640	0.023813	0.022932	0.0246
2023-01-09	0.004089	-0.015112	0.009050	-0.025909	-0.004132	0.009736	0.051753	-0.012214	-0.000567	0.0593

```
portfolio_returns = returns.mean(axis=1)
portfolio_volatility = portfolio_returns.std()

print("Portfolio Volatility:", round(portfolio_volatility, 4))
```

```
Portfolio Volatility: 0.0092
```

```
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

start_date = "2023-01-01"
end_date = "2024-01-01"

stocks = ["AAPL","MSFT","JPM","JNJ","XOM","PG","TSLA","DIS","BAC","NVDA","SPY"]

# Stock data
data = yf.download(stocks, start=start_date, end=end_date)

# Market data (S&P 500)
market_data = yf.download("^GSPC", start=start_date, end=end_date)

market_data.head()
```

Price	Close	High	Low	Open	Volume
Ticker	^GSPC	^GSPC	^GSPC	^GSPC	^GSPC
Date					
2023-01-03	3824.139893	3878.459961	3794.330078	3853.290039	3959140000
2023-01-04	3852.969971	3873.159912	3815.770020	3840.360107	4414080000
2023-01-05	3808.100098	3839.739990	3802.419922	3839.739990	3893450000
2023-01-06	3895.080078	3906.189941	3809.560059	3823.370117	3923560000
2023-01-09	3892.090088	3950.570068	3890.419922	3910.820068	4311770000

```
portfolio_returns = returns.mean(axis=1)
# Use portfolio_returns prices already prepared earlier
portfolio_prices = (1 + portfolio_returns).cumprod()

# Convert to NumPy array for efficiency
price_array = portfolio_prices.values

# Daily log returns
log_returns = np.log(price_array[1:] / price_array[:-1])

# Convert back to Series for convenience
log_returns = pd.Series(log_returns, index=portfolio_prices.index[1:])


log_returns.head()
```

```
Date
2023-01-04         NaN
2023-01-05   -0.010405
2023-01-06    0.020933
2023-01-09    0.005205
2023-01-10    0.005893
dtype: float64
```

```
mu = log_returns.mean()
sigma = log_returns.std()

trading_days = 252
simulations = 10000
```

```
np.random.seed(42)

# Random shocks
random_shocks = np.random.normal(
    loc=mu,
    scale=sigma,
    size=(trading_days, simulations)
)

# Cumulative log returns
cumulative_log_returns = random_shocks.cumsum(axis=0)

# Convert to price paths
S0 = portfolio_prices.iloc[-1]

simulated_prices = S0 * np.exp(cumulative_log_returns)
```

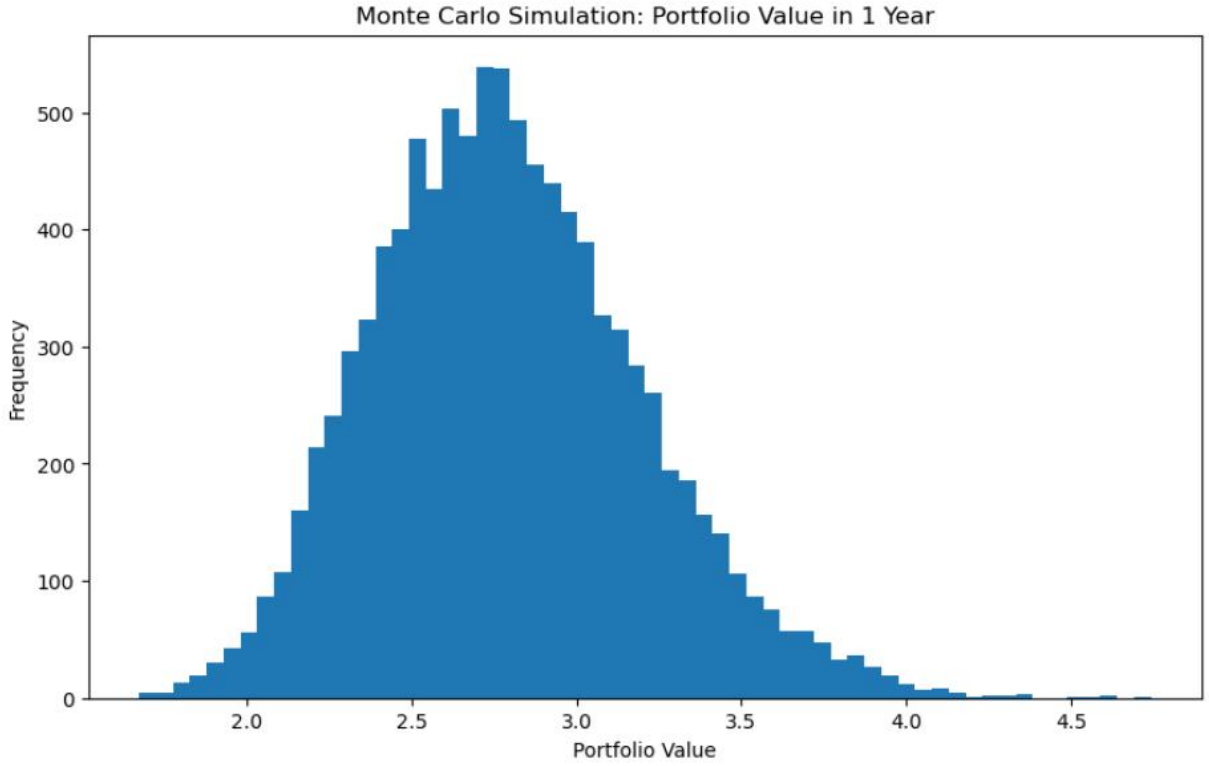
```
# Final portfolio values after 1 year
final_values = simulated_prices[-1]
final_values[:5]
```

```
array([3.55562202, 3.16633102, 2.74433811, 2.81945017, 3.01491782])
```

```
expected_value = final_values.mean()
median_value = np.median(final_values)
worst_case = np.percentile(final_values, 5)
best_case = np.percentile(final_values, 95)

expected_value, median_value, worst_case, best_case
```

```
(2.7921543482188245, 2.7646486003652857, 2.177817983425368, 3.504622574548292)
```

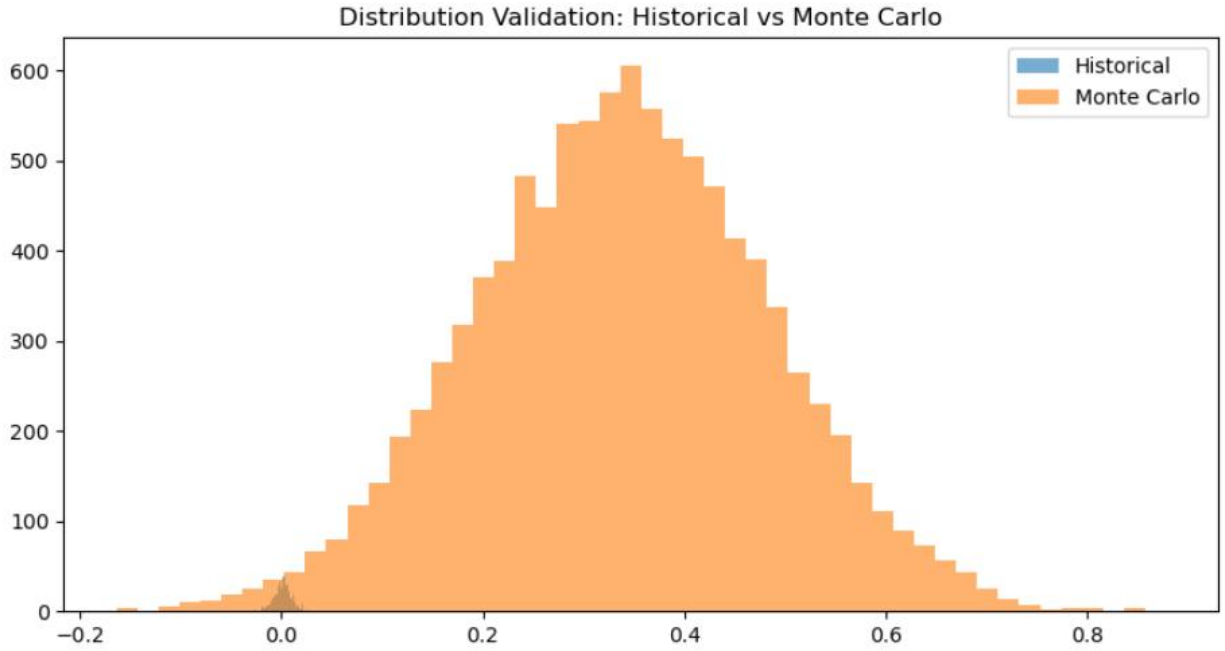
```
plt.figure(figsize=(10,6))
plt.hist(final_values, bins=60)
plt.title("Monte Carlo Simulation: Portfolio Value in 1 Year")
plt.xlabel("Portfolio Value")
plt.ylabel("Frequency")
plt.show()
```



```python
from scipy.stats import skew, kurtosis

stats_validation = pd.DataFrame({
    "Historical": [
        skew(log_returns),
        kurtosis(log_returns, fisher=True)
    ],
    "Monte Carlo": [
        skew(np.log(final_values / S0)),
        kurtosis(np.log(final_values / S0), fisher=True)
    ]
}, index=["Skewness", "Kurtosis"])

stats_validation
```

	Historical	Monte Carlo
Skewness	NaN	-0.017699
Kurtosis	NaN	-0.067430

```python
plt.figure(figsize=(10,5))
plt.hist(log_returns, bins=50, alpha=0.6, label="Historical")
plt.hist(np.log(final_values / S0), bins=50, alpha=0.6, label="Monte Carlo")
plt.title("Distribution Validation: Historical vs Monte Carlo")
plt.legend()
plt.show()
```



```
stress_scenarios = {
    "Mild Correction (-10%)": -0.10,
    "Market Crash (-30%)": -0.30,
    "Global Crisis (-50%)": -0.50
}
```

```
current_value = portfolio_prices.iloc[-1]

stress_results = {
    scenario: current_value * (1 + shock)
    for scenario, shock in stress_scenarios.items()
}

stress_results
```

```
{'Mild Correction (-10%)': 1.7734749139308157,
 'Market Crash (-30%)': 1.3793693775017455,
 'Global Crisis (-50%)': 0.9852638410726754}
```

```
worst_1 = portfolio_returns.min()
worst_5 = np.percentile(portfolio_returns, 5)
worst_1pct = np.percentile(portfolio_returns, 1)

worst_1, worst_5, worst_1pct
```

```
(-0.029946506658908915, nan, nan)
```

```
historical_stress = {
    "Worst Day": current_value * (1 + worst_1),
    "5th Percentile Loss": current_value * (1 + worst_5),
    "1st Percentile Loss": current_value * (1 + worst_1pct)
}

historical_stress
```

```
{'Worst Day': 1.9115172617904206,
 '5th Percentile Loss': nan,
 '1st Percentile Loss': nan}
```

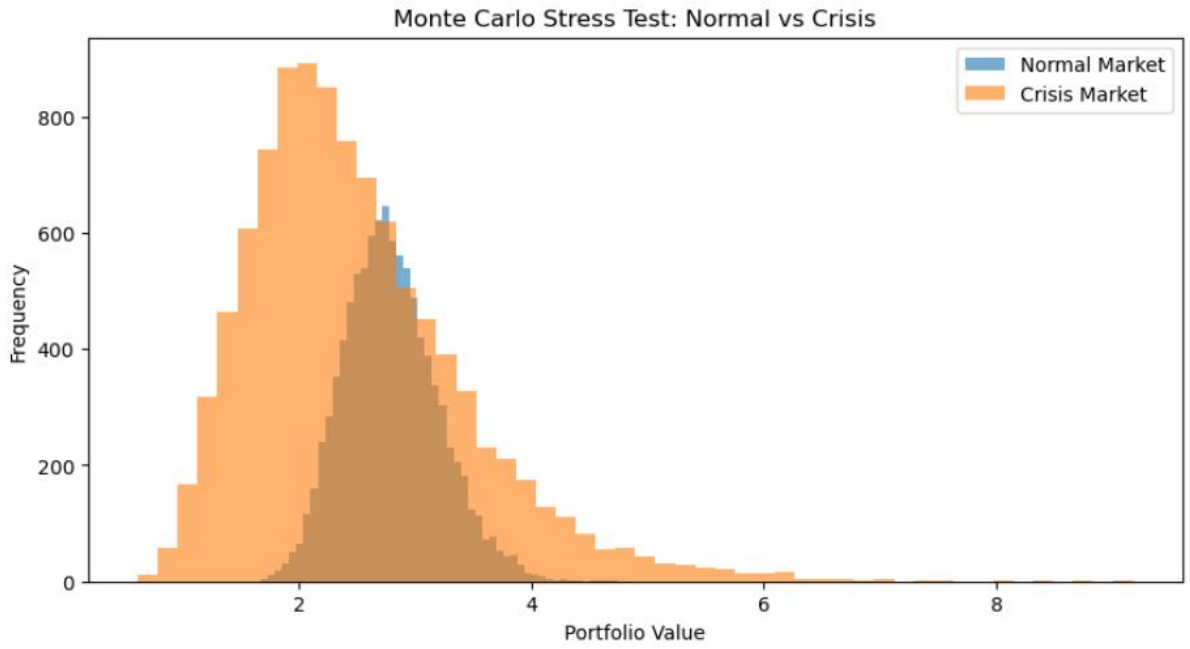
```
crisis_sigma = sigma * 2.5
crisis_mu = mu * 0.5
```

```
np.random.seed(99)

crisis_shocks = np.random.normal(
    loc=crisis_mu,
    scale=crisis_sigma,
    size=(252, 10000)
)

crisis_cum_log_returns = crisis_shocks.cumsum(axis=0)
crisis_prices = current_value * np.exp(crisis_cum_log_returns)

crisis_final_values = crisis_prices[-1]
```

```python
crisis_summary = {
    "Mean Value": crisis_final_values.mean(),
    "Median Value": np.median(crisis_final_values),
    "5th Percentile": np.percentile(crisis_final_values, 5),
    "1st Percentile": np.percentile(crisis_final_values, 1)
}

crisis_summary
```

```
{'Mean Value': 2.4964721296053285,
 'Median Value': 2.3298174310209987,
 '5th Percentile': 1.2784134962282288,
 '1st Percentile': 1.0056877999640241}
```

```python
plt.figure(figsize=(10,5))
plt.hist(final_values, bins=50, alpha=0.6, label="Normal Market")
plt.hist(crisis_final_values, bins=50, alpha=0.6, label="Crisis Market")
plt.title("Monte Carlo Stress Test: Normal vs Crisis")
plt.xlabel("Portfolio Value")
plt.ylabel("Frequency")
plt.legend()
plt.show()
```



```
crisis_drawdowns = []

for i in range(crisis_prices.shape[1]):
    path = crisis_prices[:, i]
    peak = np.maximum.accumulate(path)
    drawdown = (path - peak) / peak
    crisis_drawdowns.append(drawdown.min())

np.percentile(crisis_drawdowns, [1, 5, 50])
```

```
array([-0.57093395, -0.48854511, -0.27957275])
```

```
stress_table = pd.DataFrame({
    "Scenario": list(stress_results.keys()) + list(historical_stress.keys()),
    "Portfolio Value": list(stress_results.values()) + list(historical_stress.values())
})

stress_table
```

	Scenario	Portfolio Value
0	Mild Correction (-10%)	1.773475
1	Market Crash (-30%)	1.379369
2	Global Crisis (-50%)	0.985264
3	Worst Day	1.911517
4	5th Percentile Loss	NaN
5	1st Percentile Loss	NaN

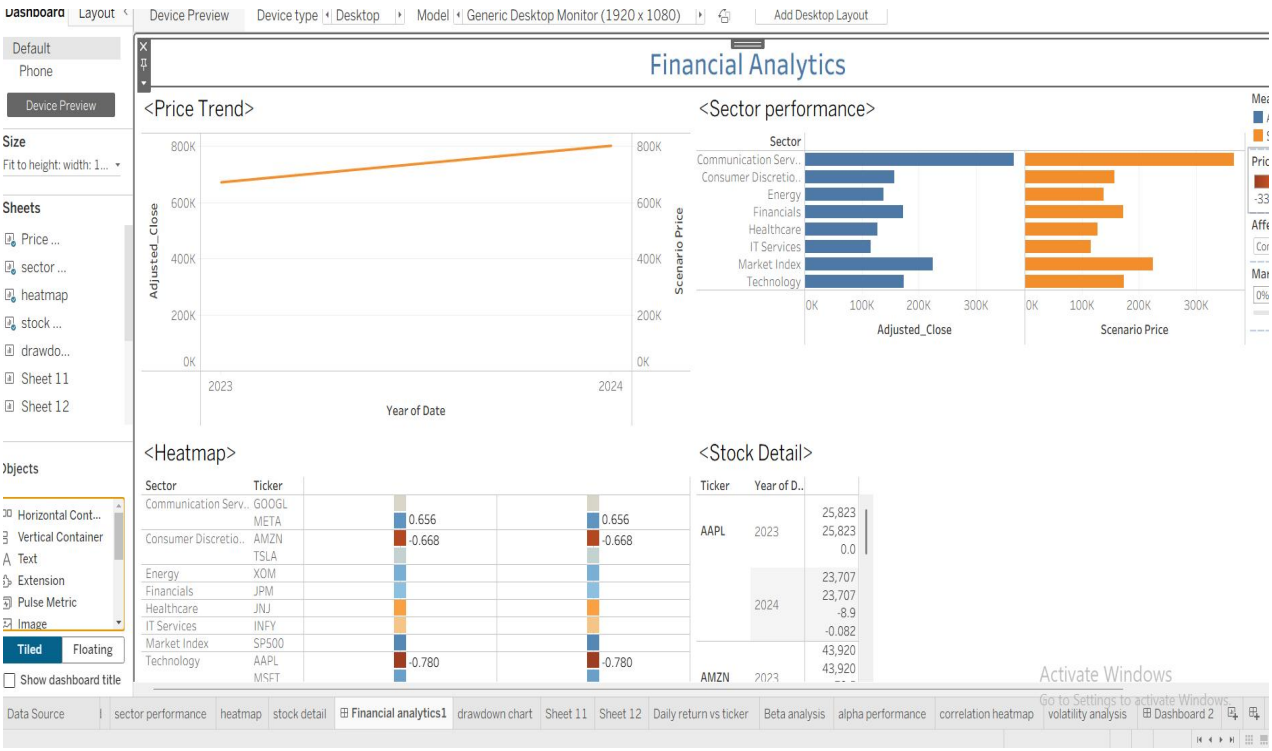
```
stress_loss_pct = {
    scenario: (value - current_value) / current_value
    for scenario, value in stress_results.items()
}

stress_loss_pct
```

```
{'Mild Correction (-10%)': -0.1,
 'Market Crash (-30%)': -0.30000000000000004,
 'Global Crisis (-50%)': -0.5}
```

Week 3: Visual Storytelling (Tableau)

Key Development Tasks: Connect Tableau directly to the cleaned, processed CSV or SQL output. Build dynamic "What If" parameters (e.g., allowing the user to input, "What if the Tech sector drops 10%?") that instantly adjust the risk calculation.

Interactivity Check: Verify that the "What If" parameters update all relevant visuals and calculations instantly without lag.



Week 4: Finalization

Tasks:Automate the market data refresh process. Create a dedicated Executive Summary tab focusing only on the highest-level KPIs (Current VaR, Max Drawdown).